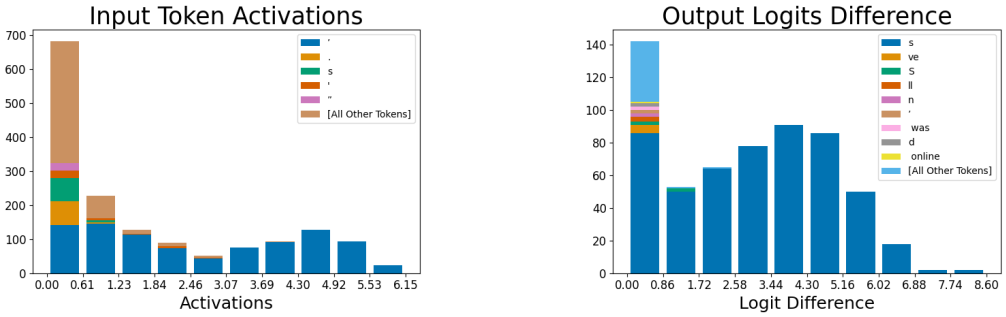

Figure 4: Histogram of token counts for dictionary feature 556. (Left) For all datapoints that activate dictionary feature 556, we show the count of each token in each activation range. The majority of activations are apostrophes, particularly for higher activations. Notably the lower activating tokens are conceptually similar to apostrophes, such as other punctuation. (Right) We show which token predictions are suppressed by ablating the feature, as measured by the difference in logits between the ablated and unablated model. We find that the token whose prediction decreases the most is the "s" token. Note that there are 12k logits negatively effected, but we set a threshold of 0.1 for visual clarity.

To better illustrate the monosemanticity of certain dictionary features, we plot the histogram of activations across token activations. This technique only works for dictionary features that activate for a small set of tokens. We find dictionary features that only activate on apostrophes (Figure 4); periods; the token " the"; and newline characters. The apostrophe feature in Figure 4 stands in contrast to the default basis for the residual stream, where the dimension that most represents an apostrophe is displayed in Figure 11 in Appendix D.1; this dimension is polysemantic since it represents different information at different activation ranges.

Although the dictionary feature discussed in the previous section activates only for apostrophes, it does not activate on *all* apostrophes. This can be seen in Figures 14 and 15 in Appendix D.2, showing two other apostrophe-activating dictionary features, but for different contexts (such as "[I/We/They]'ll" and "[don/won/wouldn]'t"). Details for how we searched and selected for dictionary features can be found in Appendix D.3.

5.2 Output: Dictionary Features have Intuitive Effects on the Logits

In addition to looking at which tokens activate the dictionary feature, we investigate how dictionary features affect the model's output predictions for the next token by ablating the feature from the residual stream[5]. If our dictionary feature is interpretable, subtracting its value from the residual stream should have a logical effect on the predictions of the next token. We see in Figure 4 (Right) that the effect of removing the apostrophe feature mainly reduces the logit for the following "s". This matches what one would expect from a dictionary feature that detects apostrophes and is used by the model to predict the "s" token that would appear immediately after the apostrophe in possessives and contractions like "let's".

5.3 Intermediate Features: Dictionary Features Allow Automatic Circuit Detection

We can also understand dictionary features in relation to the upstream and downstream dictionary features: given a dictionary feature, which dictionary features in previous layers cause it to activate, and which dictionary features in later layers does it cause to activate?

To automatically detect the relevant dictionary features, we choose a target dictionary feature such as layer 5's feature for tokens in parentheses which predicts a closing parentheses (Figure 5). For this target dictionary feature, we find its maximum activation M across our dataset, then sample 20

[5]Specifically we use less-than-rank-one ablation, where we lower the activation vector in the direction of the feature only up to the point where the feature is no longer active.

contexts that cause the target feature to activate in the range $[M/2, M]$. For each dictionary feature in the previous layer, we rerun the model while ablating this feature and sort the previous-layer features by how much their ablation decreased the target feature. If desired, we can then recursively apply this technique to the dictionary features in the previous layer with a large impact. The results of this process form a causal tree, such as Figure 5.

Being the last layer, layer 5's role is to output directions that directly correspond to tokens in the unembedding matrix. In fact, when we unembed feature 5_{2027}, the top-tokens are all closing parentheses variations. Intuitively, previous layers will detect all situations that precede closing parentheses, such as dates, acronyms, and phrases.

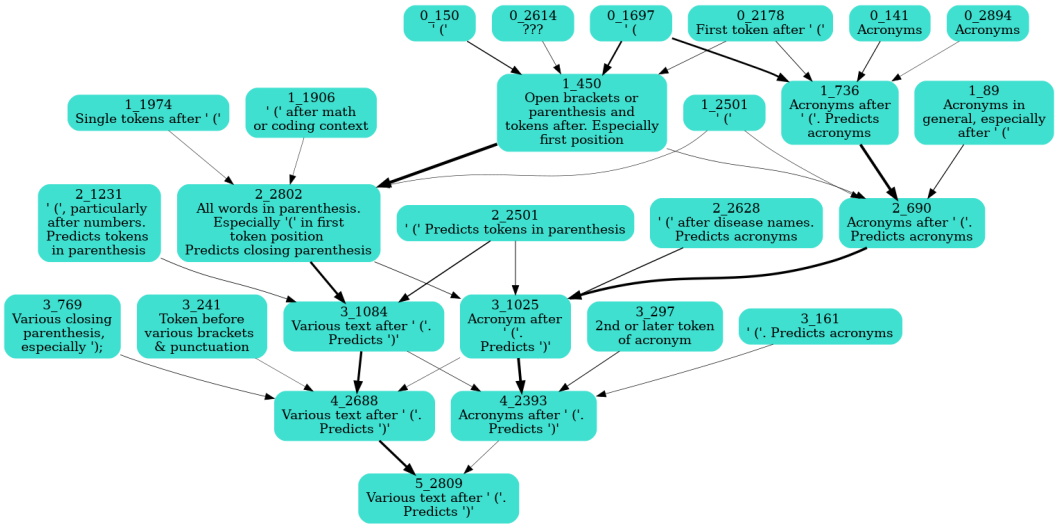

Figure 5: Circuit for the closing parenthesis dictionary feature, with human interpretations of each feature shown. Edge thickness indicates the strength of the causal effect between dictionary features in successive residual stream layers, as measured by ablations. Many dictionary features across layers correspond to similar real-world features and often point in similar directions in activation space, as measured by cosine similarity.

6 Discussion

6.1 Related Work

A limited number of previous works have learned dictionaries of sparsely-activating features in pretrained models, including Yun et al. (2021) and Sharkey et al. (2023), the latter of which motivated this work. However, similar methods have been applied in other domains, in particular in understanding neurons in the visual cortex (Olshausen & Field, 2004; Wright & Ma, 2022).

In contrast to our approach, where we try to impose sparsity after training, many previous works have encouraged sparsity in neural networks via changes to the architecture or training process. These approaches include altering the attention mechanism (Correia et al., 2019), adding ℓ^1 penalties to neuron activations (Kasioumis et al., 2021; Georgiadis, 2019), pruning neurons (Frankle & Carbin, 2018), and using the softmax function as the non-linearity in the MLP layers (Elhage et al., 2022a). However, training a state-of-the-art foundation model with these additional constraints is difficult (Elhage et al., 2022a), and improvements to interpretability are not always realized (Meister et al., 2021).

6.2 Limitations and Future Work

While we have presented evidence that our dictionary features are interpretable and causally important, we do not achieve 0 reconstruction loss (Equation 4), indicating that our dictionaries fail to capture all the information in a layer's activations. We have also confirmed this by measuring

the perplexity of the model's predictions when a layer is substituted with its reconstruction. For instance, replacing the residual stream activations in layer 2 of Pythia-70M with our reconstruction of those activations increases the perplexity on the Pile (Gao et al., 2020) from 25 to 40. To reduce this loss of information, we would like to explore other sparse autoencoder architectures and to try minimizing the change in model outputs when replacing the activations with our reconstructed vectors, rather than the reconstruction loss. Future efforts could also try to improve feature dictionary discovery by incorporating information about the weights of the model or dictionary features found in adjacent layers into the training process.

Our current methods for training sparse autoencoders are best suited to the residual stream. There is evidence that they may be applicable to the MLPs (see Appendix C), but the training pipeline used to train the dictionaries in this paper is not able to robustly learn overcomplete bases in the intermediate layers of the MLP. We're excited by future work investigating what changes can be made to better understand the computations performed by the attention heads and MLP layers, each of which poses different challenges.

In Section 4, we show that for the IOI task, behaviour is dependent on a relatively small number of features. Because our dictionary is trained in a task-agnostic way, we expect this result to generalize to similar tasks and behaviours, but more work is needed to confirm this suspicion. If this property generalizes, we would have a set of features which allow for understanding many model behaviours using just a few features per behaviour. We would also like to trace the causal dependencies between features in different layers, with the overarching goal of providing a lens for viewing language models under which causal dependencies are sparse. This would hopefully be a step towards the eventual goal of building an end-to-end understanding of how a model computes its outputs.

6.3 Conclusion

Sparse autoencoders are a scalable, unsupervised approach to disentangling language model network features from superposition. Our approach requires only unlabelled model activations and uses orders of magnitude less compute than the training of the original models. We have demonstrated that the dictionary features we learn are more interpretable by autointerpretation, letting us pinpoint the features responsible for a given behaviour more finely, and are more monosemantic than comparable methods. This approach could facilitate the mapping of model circuits, targeted model editing, and a better understanding of model representations.

An ambitious dream in the field of interpretability is enumerative safety (Elhage et al., 2022b): producing a human-understandable explanation of a model's computations in terms of a complete list of the model's features and thereby providing a guarantee that the model will not perform dangerous behaviours such as deception. We hope that the techniques we presented in this paper also provide a step towards achieving this ambition.

Acknowledgments

We would like to thank the OpenAI Researcher Access Program for their grant of model credits for the autointerpretation and CoreWeave for providing EleutherAI with the computing resources for this project. We also thank Nora Belrose, Arthur Conmy, Jake Mendel, and the OpenAI Automated Interpretability Team (Jeff Wu, William Saunders, Steven Bills, Henk Tillman, and Daniel Mossing) for valuable discussions regarding the design of various experiments. We thank Wes Gurnee, Adam Jermyn, Stella Biderman, Leo Gao, Curtis Huebner, Scott Emmons, and William Saunders for their feedback on earlier versions of this paper. Thanks to Delta Hessler for proofreading. LR is supported by the Long Term Future Fund. RH is supported by an Open Philanthropy grant. HC was greatly helped by the MATS program, funded by AI Safety Support.

References

Stella Biderman, Hailey Schoelkopf, Quentin Gregory Anthony, Herbie Bradley, Kyle O'Brien, Eric Hallahan, Mohammad Aflah Khan, Shivanshu Purohit, USVSN Sai Prashanth, Edward Raff, et al. Pythia: A suite for analyzing large language models across training and scaling. In *International Conference on Machine Learning*, pp. 2397–2430. PMLR, 2023.